

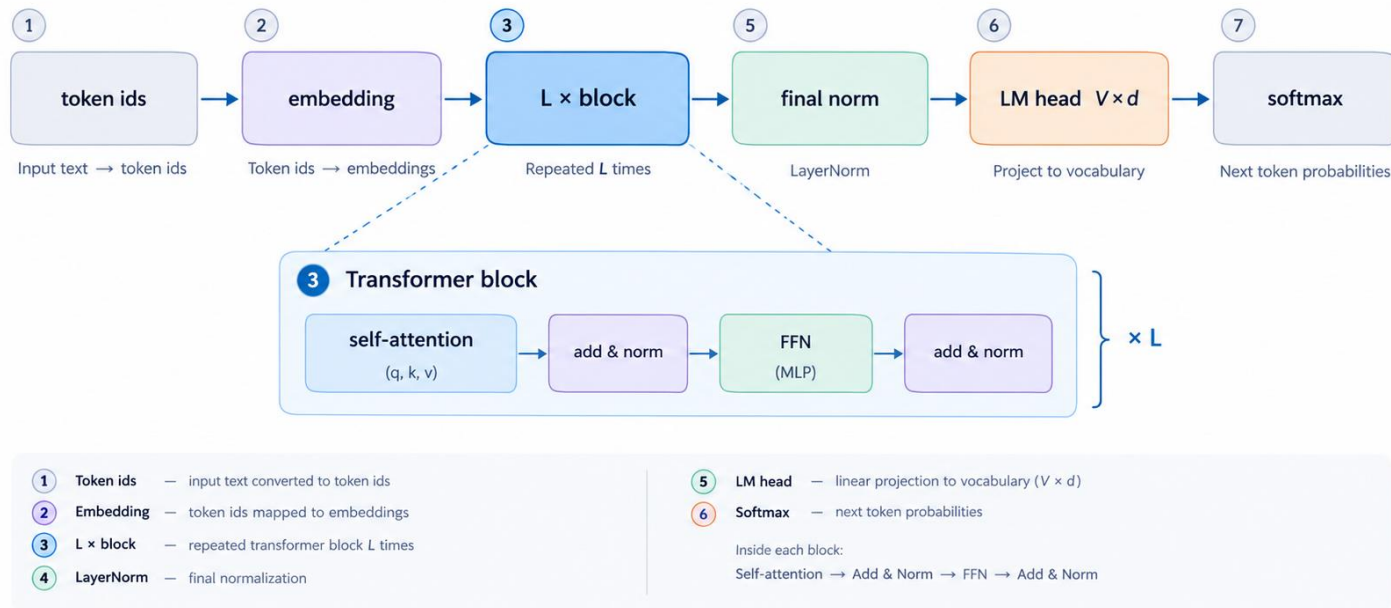
LECTURE 2

FFN, MoE, Residual Stream & Novel Architectures

Gleb Molodtsov · 14 September 2026



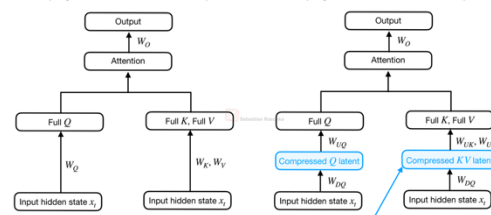
Lecture 1 followed one token to the end of attention



Three dials inside attention, and you have turned all three

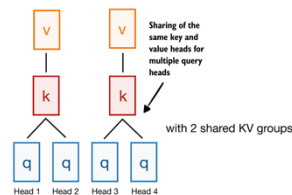
The dial	What was tried	What it buys	What it costs
How many KV heads	MHA $\rightarrow$ GQA $\rightarrow$ MLA	KV per token	cache size, not quality
Who may be looked at	full $\rightarrow$ sliding window $\rightarrow$ sparse	context length	what falls out of view
Where position enters	absolute $\rightarrow$ RoPE $\rightarrow$ NoPE	extrapolation	θ base, YaRN, or nothing

Regular Multi-head Attention (MHA)
(only one attention head shown)



This compressed representation is stored in the KV cache

Grouped query attention (GQA)

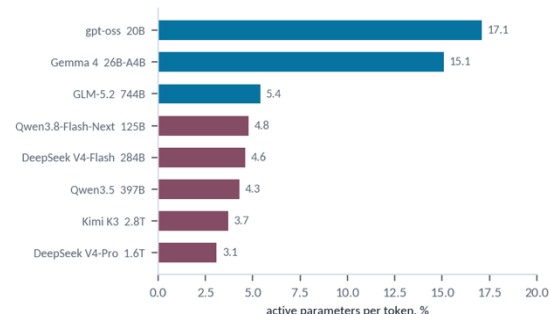


WHY THIS, WHY NOW

Four fresh releases, and how big is never one number



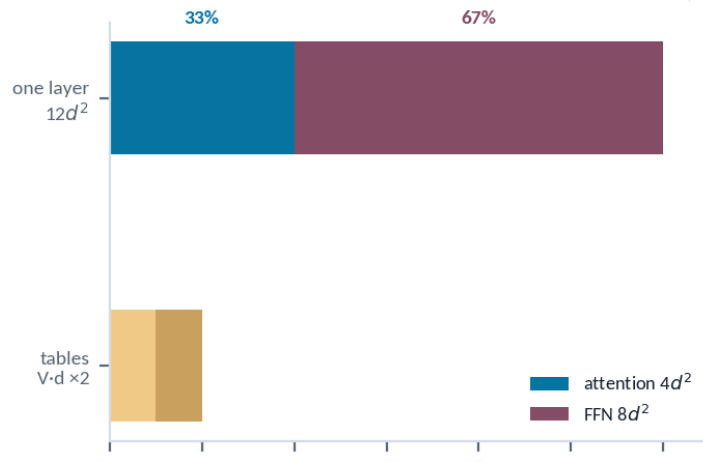
Model	Date	Total	Active	How the FFN is built
DeepSeek V4.1-Flash	10.09.26	552B + 196B	8B / 16B	384 routed + 1 shared, top-6
Kimi K3	27.07.26	2.8T	104B	896 routed + 2 shared, 16 active
GLM-5.3-Flash	08.26	320B	18B	scaled down from 744B-A40B
Qwen3.8-Flash-Next	08.26	180B	6B	512 experts of width 640, top-10 + 1
Qwen3.8-27B	14.08.26	27.8B	27.8B	dense, no experts at all



Attention costs time; parameters sit in the FFN and the head

Per layer: $4d^2$ in attention, $8d^2$ in the FFN

- Attention is quadratic in tokens — that is where the **time** goes
- The FFN is two thirds of every block — that is where the **parameters** go



Six blocks, and the break after the second



01 FFN: what the wide matrix is for

02 MoE: one FFN becomes many

03 Normalization: what decides whether a run converges

04 The residual stream: what the highway carries

05 LM head: the price of a big vocabulary

06 Four tech reports, read by axis



01

- **FFN: MLP $\rightarrow$ SwiGLU $\rightarrow$ MoE**

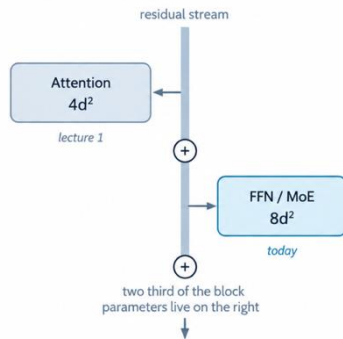
Two thirds of the block, and all of the sparsity of the frontier



Most of the model is FFN, and it never mixes positions

THE SHAPE

- $\text{up}(d \rightarrow 4d) \rightarrow \text{nonlinearity} \rightarrow \text{down}(4d \rightarrow d)$
- $8d^2$ per layer against attention's $4d^2$
- Applied to every position independently, with shared weights

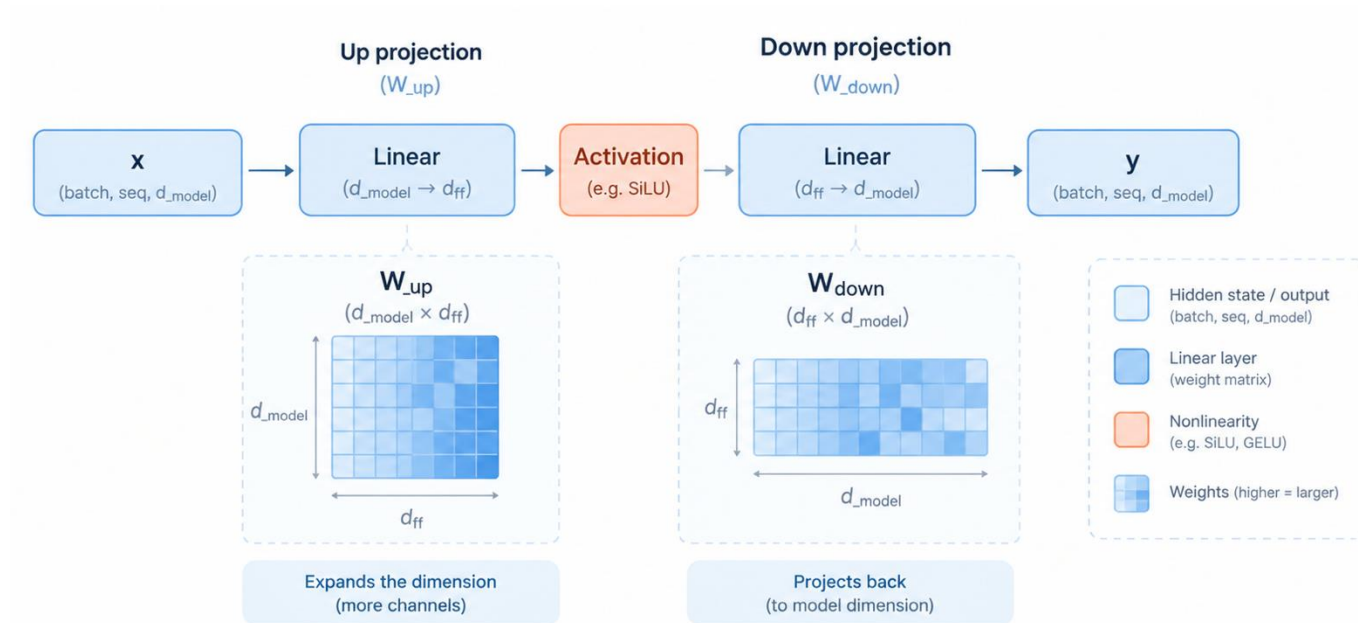


WHY THAT MATTERS

- Attention moves information between positions
- FFN transforms one



Up builds detectors; down decides what gets written back



Read the FFN as memory

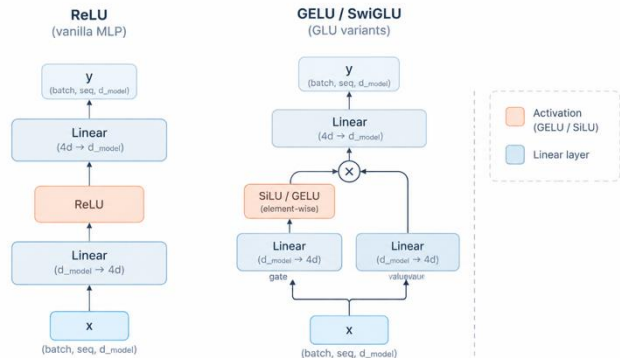
Rows of the wide matrix behave like **keys** matched against the token; **columns** of the narrow one like the **values** written back.

- Individual keys fire on patterns
- **Deep** layers key on **semantics**, shallow ones on **surface form**
- The **FFN is memory**, enlarging it is cheaper than enlarging attention
- That is the whole motivation for the sparsity

The activation changed twice, and once without a reason

ReLU in the 2017 Transformer, GELU through the BERT and GPT-2 era, SwiGLU in almost everything since PaLM.

- ReLU → GELU: a smooth gate, better gradients near zero
- GELU → SwiGLU: a second branch that multiplies, not just passes
- The GLU-variants paper credits «divine benevolence» — there is still no mechanism
- It survives because it wins reproducibly at an equal parameter budget



Shazeer, GLU Variants Improve Transformer, 2020 - arXiv:2002.05202



Three matrices, a narrower d_{ff} , and still exactly $8d^2$

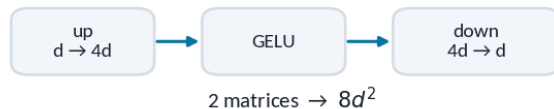
THE ARITHMETIC

- Gate branch and value branch, multiplied elementwise, then projected down
- Three matrices $d \times d_{ff}$ instead of two, so d_{ff} drops from $4d$ to $8/3 \cdot d$
- The budget lands exactly: $3 \cdot d \cdot (8/3 \cdot d) = 8d^2$, same as vanilla

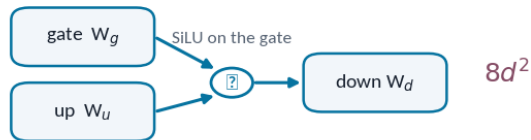
WHAT THE GATE BUYS

- The value branch is scaled per channel by data, not by a fixed curve
- Same parameters and FLOPs, one more elementwise product
- One more weight matrix to stream, so the memory traffic does grow

vanilla FFN, $d_{ff} = 4d$



SwiGLU, $d_{ff} = \frac{8}{3}d$



3 matrices, narrower d_{ff} — same budget: 134.2M at $d = 4096$



02

- **MoE: one FFN becomes many**



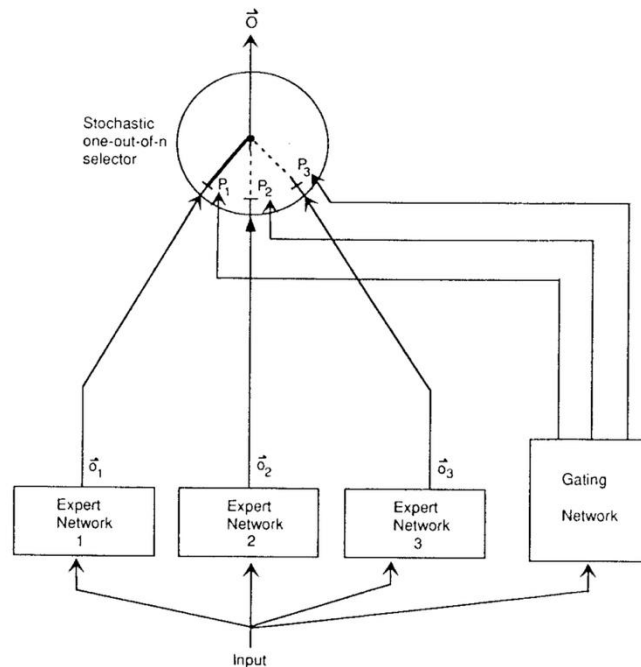
An ensemble averages; a mixture divides the work

THE ENSEMBLE, AND WHY IT IS NOT THIS

- Train M models, run all M , average the outputs
- The gain is variance reduction — errors are partly independent
- Cost scales with M : you pay for every member on every input

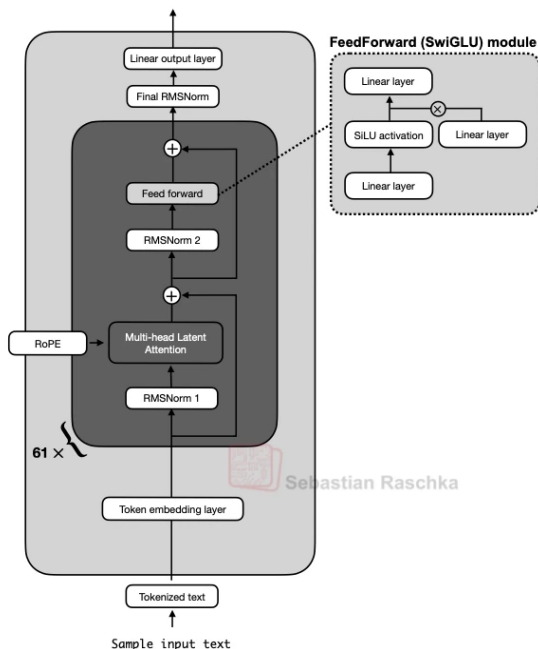
THE MIXTURE, 1991

- Jacobs, Jordan, Nowlan & Hinton: adaptive mixtures of local experts
- A gating network splits the input space; each expert owns a region
- Now the goal is division of labor, and you run one member, not M

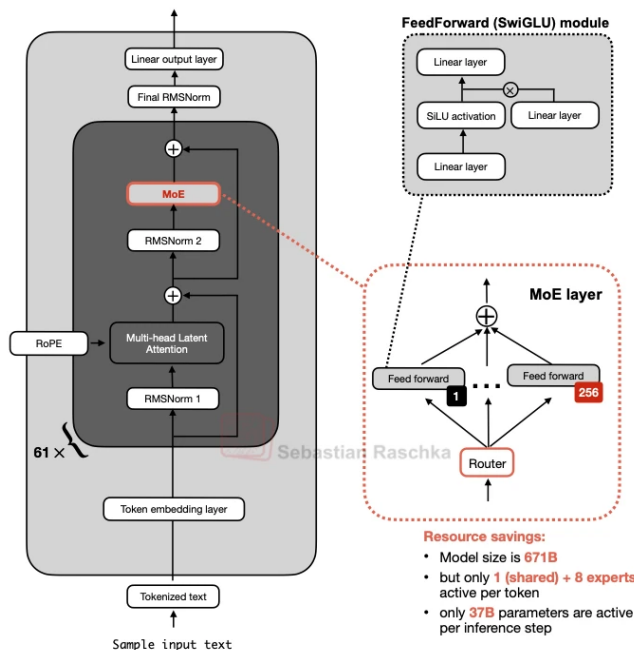


An ensemble averages; a mixture divides the work

Architecture
without MoE (“dense”)



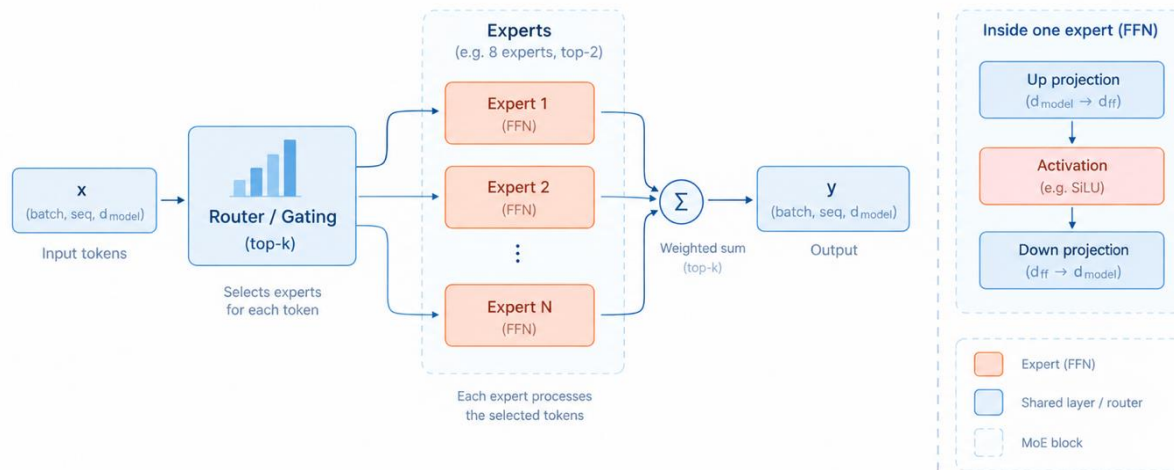
DeepSeek V3/R1
with MoE (“sparse”)



A gate in front of the FFN splits total from active

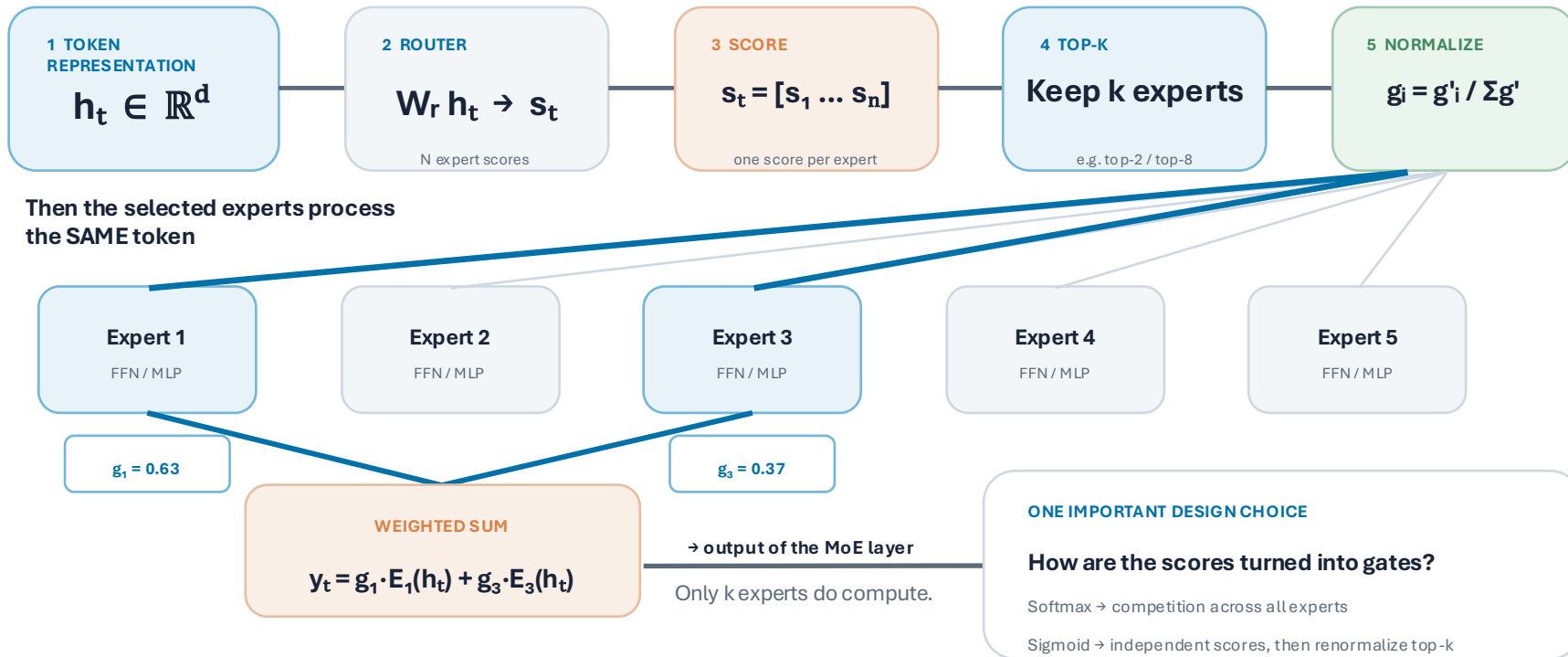
One FFN becomes N experts. Shazeer, 2017: make the gate sparse, and compute stops scaling with N.

- $h' = u + \sum_{i \in \text{top-}k} g_i \cdot \text{FFN}_i(u)$
- N grows the parameters; k fixes the compute — the two numbers come apart



How the MoE router sends each token to experts

For ONE token, the router is just: score $\rightarrow$ choose top-k $\rightarrow$ normalize $\rightarrow$ mix expert outputs



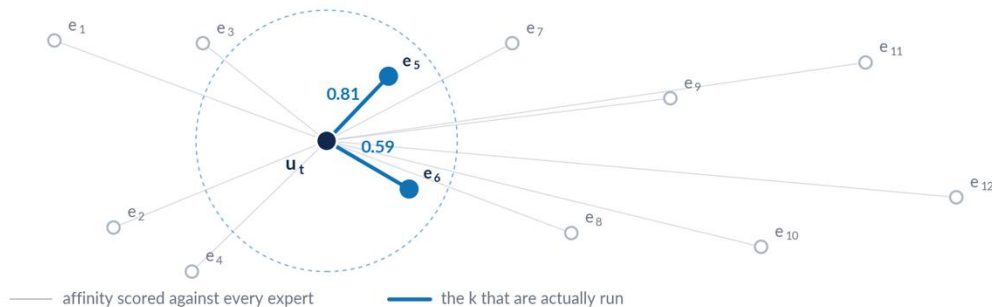
Top-k in three lines: affinity, selection, renormalization

WHAT THE ROUTER COMPUTES

- $s_{i,t} = \sigma(u_t^T e_i)$ — affinity between the token and expert i 's centroid
- $g'_{i,t} = s_{i,t}$ if $s_{i,t} \in \text{TopK}(\{s_{i,t}\}, K)$, else 0
- $g_{i,t} = g'_{i,t} / \sum_j g'_{j,t}$ — the surviving scores are renormalised

THE ONE DESIGN CHOICE UNDER IT

- σ = softmax over all N , then top-k — Mixtral, Qwen3
- σ = sigmoid per expert, then renormalise over the chosen — DeepSeek V3 onwards
- With softmax the gate scale depends on how many experts exist at all



SELECTION

e_5 $s = 0.81$ $\rightarrow$ $g = 0.58$

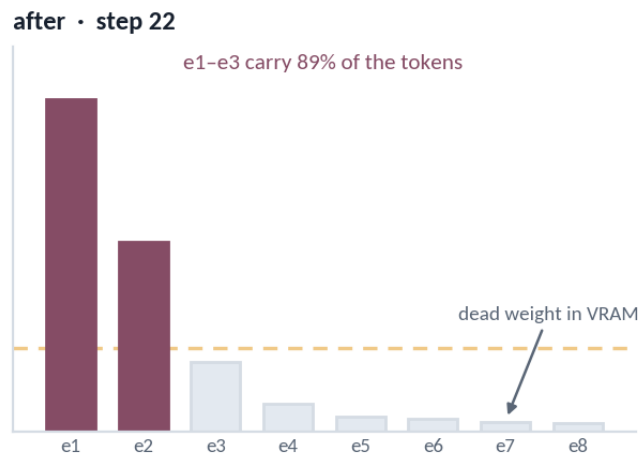
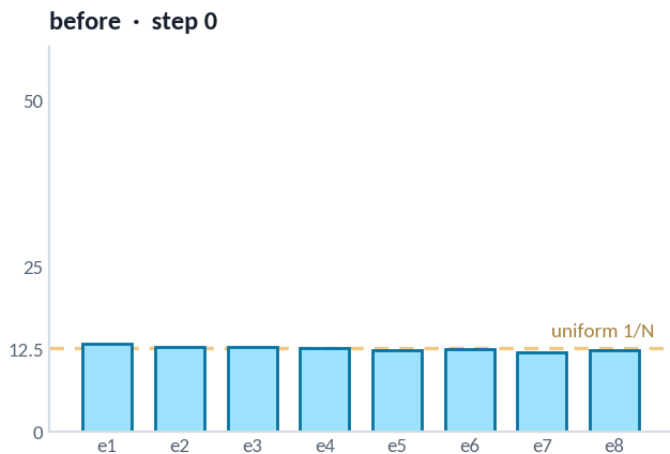
e_6 $s = 0.59$ $\rightarrow$ $g = 0.42$

renormalised over the chosen k , not over E
 the other 10 centroids stay scored but unused
 — no FFN, no FLOPs, no gradient



Train it as is and the experts collapse into a few winners

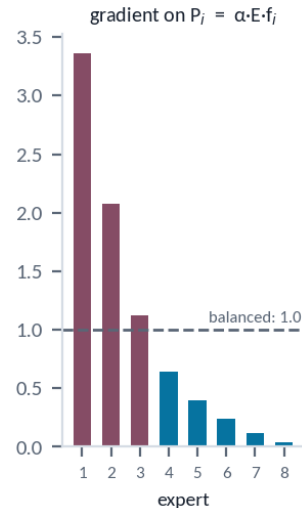
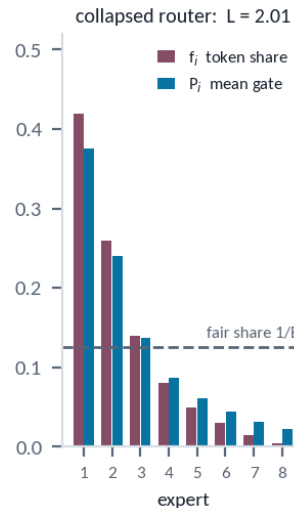
- An expert picked slightly more often **gets more updates and improves faster**
- The rest **stay undertrained**, so the router is right to avoid them
- Improving raises its affinity, so it is picked **more often**



The balance loss multiplies two things people confuse

$L_{\text{aux}} = \alpha \cdot N_E \cdot \sum_i f_i \cdot P_i$ — and the whole trick is that f and P are not the same quantity.

- f_i — what happened: the share of tokens actually routed to expert i . A hard count off the TopK indicator, no gradient
- P_i — what the router intended: its mean gating score for expert i over the batch. Soft, differentiable
- **Therefore** $\partial L / \partial P_i = \alpha \cdot N_E \cdot f_i$ — all of the gradient lands on the intention, weighted by the realised load
- **Read it as:** the more tokens an expert already took, the harder its own score is pushed down
- **Range:** perfectly balanced gives 1, full collapse onto one expert gives N_E



Five balancing losses, five pitfalls their authors name



Where	What was added	What its own authors flag
GShard, 2020	$\sum f_i P_i + \text{capacity } C \approx 2N/E$	over-reliance fakes the distribution
Switch, 2021	same loss, top-1, capacity factor	wrong CF drops far too many tokens
DeepSpeed-MoE, 2022	$\alpha \sum f_i - 1/E $	distance form, harder to tune
ST-MoE, 2022	z-loss: $(1/B) \sum (\log \sum \exp x_{ij})^2$	stability, not balance — different job
Skywork-MoE, 2024	adaptive α per layer + logit norm	λ too high sharpens the gate too far
DeepSeekMoE, 2024	expert-level α_1 and device-level α_2	two knobs instead of one

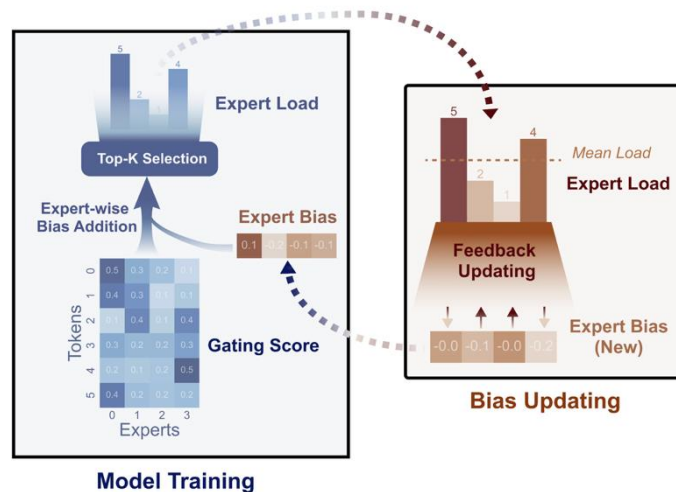
Обзор: Yihua Zhang, A Review on the Evolvement of Load Balancing Strategy in MoE LLMs, HF blog, 02.2025 — источник всех шести формул



Loss-free: move the correction out of the gradient entirely

DeepSeek-V3 adds a per-expert bias b_i to the selection only — never to the weight the expert enters the sum with.

- Select on $s_{i,t} + b_i$; combine with plain $s_{i,t}$. The bias shifts who, not how much
- $b_i \leftarrow b_i - \gamma$ if overloaded, $b_i \leftarrow b_i + \gamma$ if underloaded — outside autograd
- So the LM objective never competes with a balancing term
- Pitfall named by the authors: γ too large and the gating thrashes



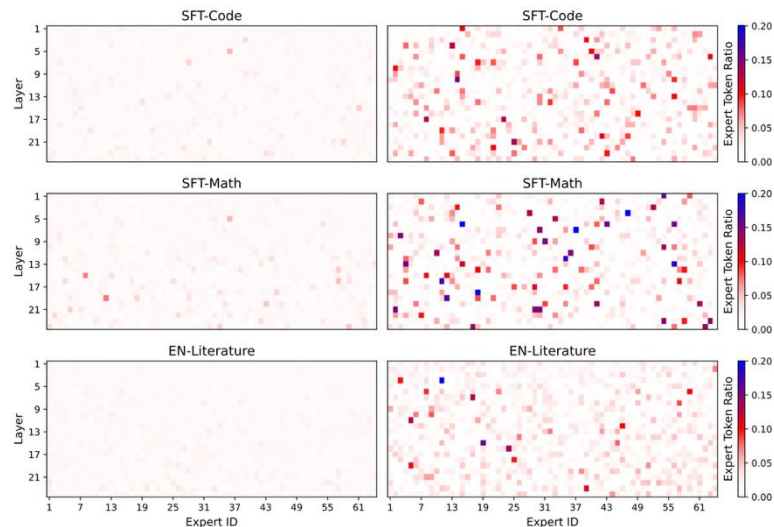
$$g_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \text{Topk}(\{s_{j,t} + b_j \mid 1 \leq j \leq N\}, K), \\ 0, & \text{otherwise.} \end{cases}$$

Wang et al., arXiv:2408.15664 • DeepSeek-V3, arXiv:2412.19437



Where you compute f_i matters more than which loss

- A code sequence then gets spread evenly over all experts — **specialization is forbidden**
- Fix: sync f_i across micro-batches and **balance over the global batch** instead
- **Domain specialization** only appears at all under global-batch balancing
- Under micro-batch balance, aux-loss and loss-free perform about the same — both worse



Qiu et al., Demons in the Detail, ACL 2025, arXiv:2501.11873, Fig 1(b)
Expert Frequency for Micro-Batch Balance (Left) and Global-Batch Balance (Right)



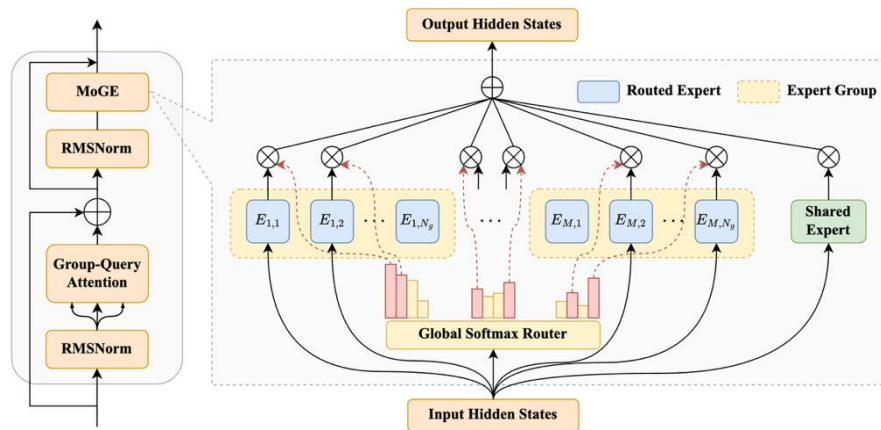
MoGE balances by construction instead of by penalty

HOW IT WORKS

- N experts are partitioned into M non-overlapping groups, one group per device
- Global softmax gives the scores; top-K' is then taken inside each group
- Every token activates the same number of experts on every device

WHAT IT COSTS

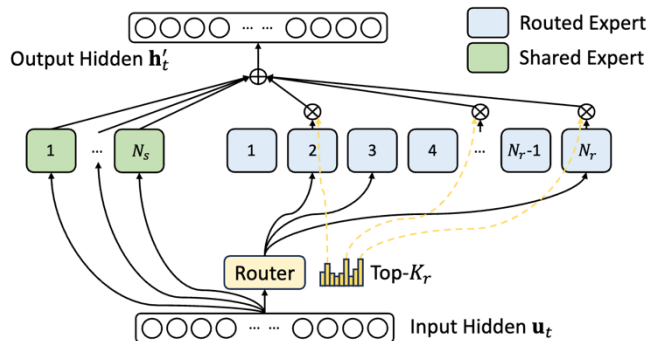
- Imbalance score is 0 by design — no penalty, no bias, no α and no γ
- But the router can no longer pick the globally best k experts
- Pangu Pro MoE, 72B total and 16B active, built for Ascend NPUs



Shared experts: everyone ships them, the evidence is mixed

THE ARGUMENT, AND DEEPSEEKMOE'S TWO KNOBS

- Always on, outside the routing: common patterns get one home instead of N copies
- The same paper also splits experts finer — both are ablated together in Fig. 3
- So «fine-grained + shared» entered the field as one package, not two results

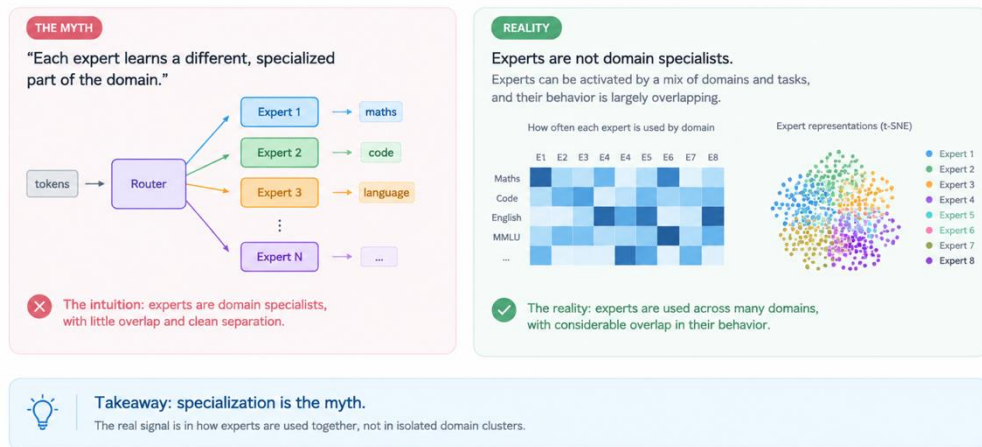


WHAT THE ABLATIONS ACTUALLY SAY

- DeepSeekMoE: clear gains. OLMoE §4.1.3: limited or negative impact
- NVIDIA, upcycling a dense model: convergence looks the same either way
- Qwen3 235B-A22B dropped it; Qwen3.6 35B-A3B brought it back a year later
- Almost all tests are binary on/off at small scale — the axis is barely measured



Balanced or not, the experts are geometry — not domains



THE MECHANISM

- The router is a linear map, so similar hidden states must pick similar experts
- Specialization is a property of the representation space, not of the routing

WHY IT RESISTS READING

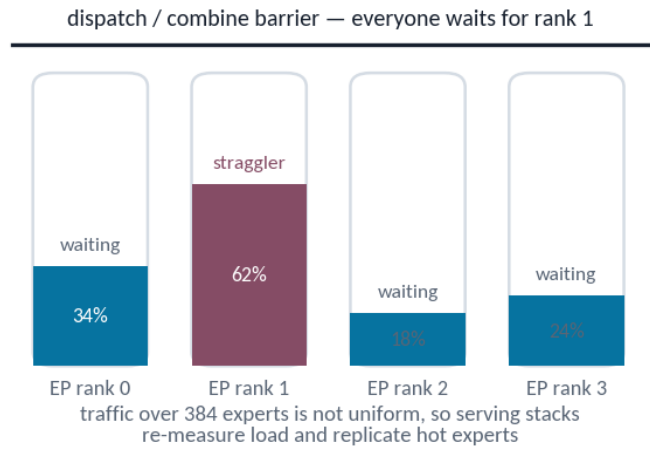
- Two models on the same question overlap ~60% — same as on unrelated questions
- Deep layers use near-identical experts for unrelated domains during prefill



Balance to train and serve, then trim for your domain

In an expert-parallel step every rank waits for the slowest. Balance is a throughput requirement, not a quality one.

- But a deployed model on one domain keeps activating a stable, sparse subset
- Few-shot expert localization: a handful of in-domain examples reveals which
- REAP scores experts by gate value and activation norm — near-lossless at 50% pruned
- Push it further and a fine-tuned single expert kept 99.3% on six tasks, 2× faster
- Same move as trimming the vocabulary in block 05: train broad, deploy narrow



EASY-EP, NeurIPS 2025 · REAP, Lasby et al., 2026 · Chen et al., arXiv:2206.00277





Sparsity is a memory-for-compute trade with a routing problem attached

552B resident · 8B multiplied · and the balance is still tuned, not solved



03

- **Normalization: what decides whether a run converges**

Not a list of variants — where exactly the norm sits relative to «+»



RMSNorm dropped the mean and lost nothing measurable

LAYERNORM, 2016

- Subtract the mean, divide by the standard deviation, then gain and bias
- Two passes over the vector and two learned vectors per norm
- Re-centering was assumed to be the part that mattered

$$y = \frac{x - \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}} * \gamma + \beta$$

RMSNORM, 2019 — AND NOW UNIVERSAL

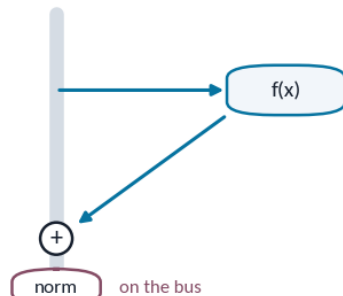
- $x / \text{rms}(x) \cdot g$ — no mean subtraction, no bias term
- Re-scaling alone reproduces the stability; re-centering was not load-bearing
- Cheaper per call and one fewer tensor to keep, at matched quality

$$\bar{a}_i = \frac{a_i}{\text{RMS}(\mathbf{a})} g_i, \quad \text{where } \text{RMS}(\mathbf{a}) = \sqrt{\frac{1}{n} \sum_{i=1}^n a_i^2}.$$



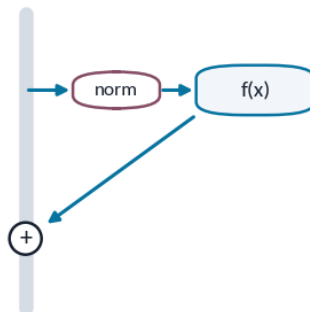
Two places for the norm, and only one keeps the sum clean

Post-LN · 2017



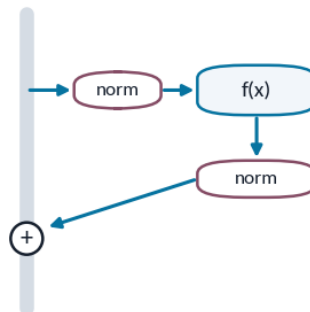
$x \leftarrow \text{Norm}(x + f(x))$
needs warmup, unstable with depth

the «+» sits at the same height in all four panels — only the norm moves
Pre-LN · default



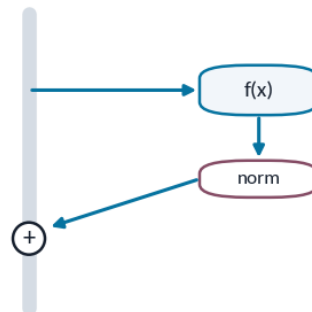
$x \leftarrow x + f(\text{Norm}(x))$
clean bus, variance grows with depth

Sandwich · Peri-LN



$x \leftarrow x + \text{Norm}(f(\text{Norm}(x)))$
Gemma 2, 3, 4 · Trinity Large

Post-flavour · in branch



$x \leftarrow x + \text{Norm}(f(x))$
OLMo 2 and OLMo 3

- Post-LN, 2017: $x_{l+1} = \text{LN}(x_l + F(x_l))$ — the norm sits on the residual path itself
- Pre-LN: $x_{l+1} = x_l + F(\text{LN}(x_l))$ — the norm moved inside the branch, the sum is untouched
- One symbol moved. Everything on the next slide follows from that move



Post-LN multiplies the LN Jacobians, Pre-LN adds them

Same LayerNorm both times. What changes is whether its Jacobian sits on the residual path or beside it.

THE SHARED FACT — LN'S JACOBIAN IS A $1/\sigma$ MULTIPLIER

$$\frac{\partial \text{LN}(x)}{\partial x} = \frac{1}{\sigma} \left(I - \frac{1}{d} \mathbf{1} \mathbf{1}^\top \right) (I - \hat{z} \hat{z}^\top) \Rightarrow \left\| \frac{\partial \text{LN}}{\partial x} \right\| \simeq \frac{1}{\sigma}, \quad \sigma = \text{std}(x)$$

POST-LN — THE JACOBIAN SITS ON THE SHORTCUT

$$x_{l+1} = \text{LN}(x_l + F(x_l))$$

$$\frac{\partial x_{l+1}}{\partial x_l} = J_l(I + F'_l)$$

$$\frac{\partial \mathcal{L}}{\partial x_l} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{k=l}^{L-1} J_k(I + F'_k)$$

- Post-LN resets the stream to $\sigma = 1$ at every block, so LN's own input has $\sigma = \sqrt{1 + \sigma_F^2}$ and every J_k contributes the same $c \approx 0.7$ at init
- Along the shortcut $(I + F'_k)$ contributes exactly I , so nothing gives c back: layer l receives $c^{(L-l)}$, i.e. $0.7^{12} \approx 0.014$ at depth 12

PRE-LN — THE JACOBIAN SITS ON THE BRANCH

$$x_{l+1} = x_l + F(\text{LN}(x_l))$$

$$\frac{\partial x_{l+1}}{\partial x_l} = I + A_l, \quad A_l = F'_l J_l$$

$$\prod_{k=l}^{L-1} (I + A_k) = I + \sum_k A_k + \sum_{i < j} A_i A_j + \dots$$

- The leading I is a route from the loss straight to layer l with no J on it. Nothing downstream can attenuate it
- σ is never reset here, so $\sigma(x_l)$ grows like $\sqrt{l}$ and $\|A_l\|$ shrinks like $1/\sqrt{l}$ — the correction terms damp themselves as the stack grows



Pre-LN's bound carries a $1/\sqrt{L}$, Post-LN's carries no L

WHERE THE $1/\sqrt{L}$ COMES FROM — THE RESIDUAL STREAM IS NEVER RENORMALISED

$$\sigma(x_l)^2 \simeq \sigma_0^2 + \sum_{k < l} \sigma_{F_k}^2 \Rightarrow \sigma(x_l) \sim \sqrt{l} \Rightarrow \|A_l\| \sim l^{-1/2} \Rightarrow \left\| \sum_{k < L} A_k \right\| = \mathcal{O}(1) \text{ for any } L$$

POST-LN, LAST LAYER

$$\left\| \frac{\partial \tilde{\mathcal{L}}}{\partial W^{2,L}} \right\|_F \leq \mathcal{O}(d\sqrt{\ln d})$$

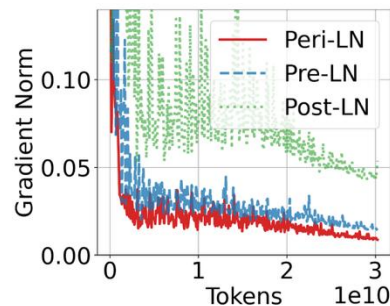
no L in it — the same size at any depth

PRE-LN, LAST LAYER

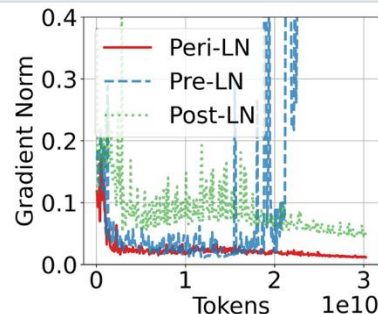
$$\left\| \frac{\partial \tilde{\mathcal{L}}}{\partial W^{2,L}} \right\|_F \leq \mathcal{O}\left(d\sqrt{\frac{\ln d}{L}}\right)$$

falls as $1/\sqrt{L}$, by the same factor at every layer

- Post-LN: no L in the bound. At init the last FFN's norm is ≈ 1.6 at 6, 8 and 12 layers alike while layer 1 is orders below — one LR cannot serve both, which is what warmup buys
- Pre-LN: the $1/\sqrt{L}$ hits every layer at once, so the profile is flat in L — hence training without warmup
- The theorem bounds one layer. The depth-balance claim rests on the measured profiles, not on this bound
- Pre-LN is not perfectly flat either: NormFormer measures the opposite tilt, early layers larger than late



(a) Gradient-norm at seed 5



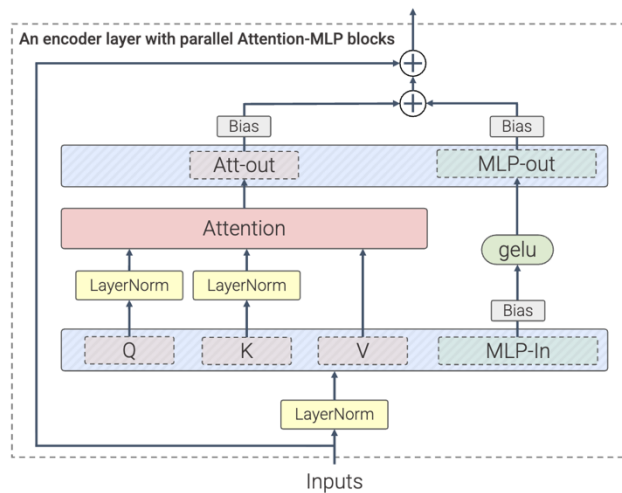
(b) Gradient-norm at seed 4



QK-norm exists because attention logits run away

Physically: bound $\|q\|$ and $\|k\|$ per head, so the dot product cannot run away.

- ViT-22B diverged near 8B: logits grew, attention went near one-hot, entropy ≈ 0
- A saturated softmax has no gradient on the logits — the head freezes on one token
- The fix: RMSNorm on Q and K before RoPE, per head — two lines in forward
- Kimi K2 and K3 clip inside the optimizer instead: QK-clip with MuonClip



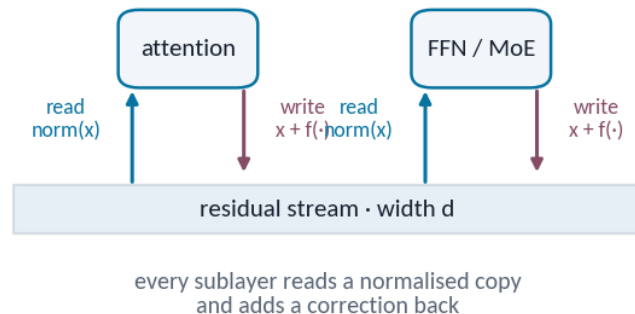
Dehghani et al., ViT-22B, arXiv:2302.05442 · OLMo 2, arXiv:2501.00656



The residual stream is a bus of fixed width

Depth became usable because contributions add up instead of composing multiplicatively.

- Every sublayer reads a normalised copy and adds a correction back
- The gradient reaches layer 1 by a path made only of additions
- Pre-LN gives an exact identity: $x_L = x_0 + \Sigma \text{ branch outputs}$
- Any two layers talk only through those d channels, and 60+ blocks compete for them



where the norm sits → block 03 · what «+» becomes → block 04

наша диаграмма · тождество проверяется численно на семинаре



04

- **The highway: residual → hyper-connections**

A year ago this axis was speculative. In 2026 three teams moved on it



What travels on the bus is directions, not neurons

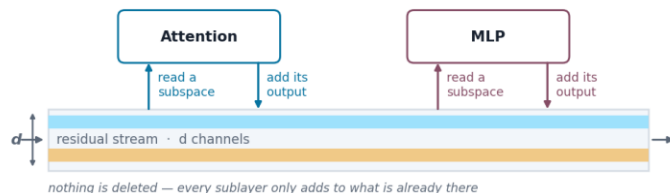
THE STRUCTURAL PICTURE

- The stream is a communication channel; sublayers **read and write** subspaces of it
- A concept corresponds to a **direction**, not to a single coordinate
- More concepts than dimensions fit, because the directions are only **near-orthogonal**

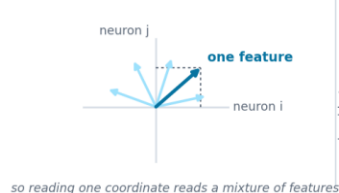
WHY THIS MATTERS TODAY

- Width d is a bandwidth budget, not just a capacity number
- Sublayers compete for the same d channels — that is the contention hyper-connections attack
- How those directions are found and edited: lecture 14, not today

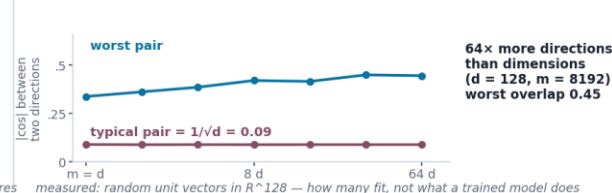
THE STREAM IS A CHANNEL



A FEATURE IS A DIRECTION



DIRECTIONS ARE ONLY NEAR-ORTHOGONAL — SO MORE THAN d FIT



Depth runs in four stages, and the names are useful

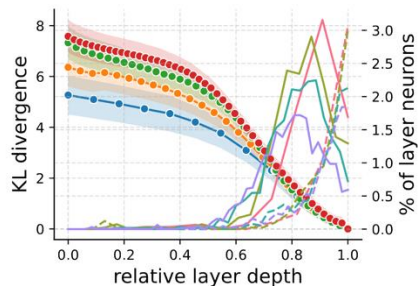


Table 1: Our Hypothesis: Universal Inference Stages

Stage	Name	Function	Observable signatures
1	Detokenization	Integrate local context to transform raw token representations into coherent entities	Catastrophic sensitivity to deletion and swapping and attention-heavy computation.
2	Feature Engineering	Iteratively build feature representation depending on token context	Little progress made towards next token prediction, but significant increase in probing accuracy and patching importance.
3	Prediction Ensembling	Convert previously constructed semantic features into plausible next token predictions using an ensemble of model components	Prediction neurons appear and output distribution begins to narrow.
4	Residual Sharpening	Sharpen the next token distribution by eliminating obsolete features that add noise to the prediction	Suppression neurons appear and output distribution narrows with a growing MLP-output norm

- Early layers act as an extended embedding: subwords are fused back into words and entities
- Middle layers build features and quietly commit — the answer is often decided well before the end
- Late layers sharpen and format, moving probability mass onto the chosen token
- The authors' own caveat: the boundaries are fuzzy and stages overlap



Middle layers are the richest and the most redundant

Two findings that sound contradictory and are not: mid-depth carries the best representation, and mid-depth survives being cut.

- Intermediate layers beat the final layer by up to 16% on 32 embedding tasks
- Autoregressive models show a mid-layer compression valley; masked models do not
- Deleting or reordering middle layers keeps 72–95% of accuracy — first and last do not
- Consequences already in production: layer pruning, early exit, and depth that is not fully used

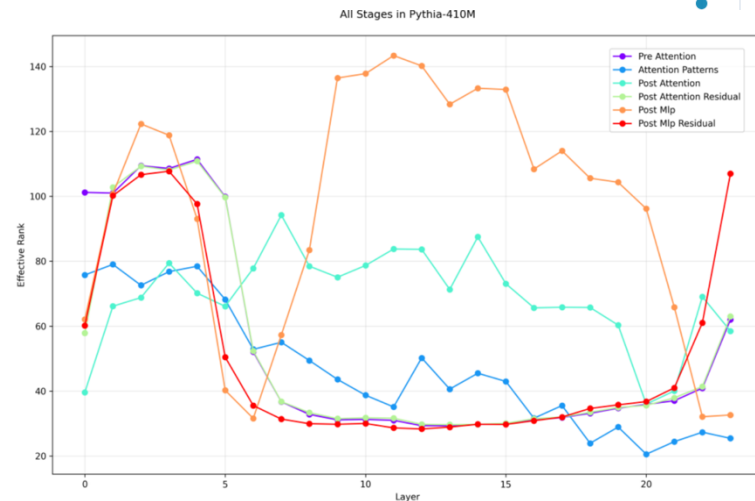


Figure 15: Behavior of effective rank at different stages within a transformer block.

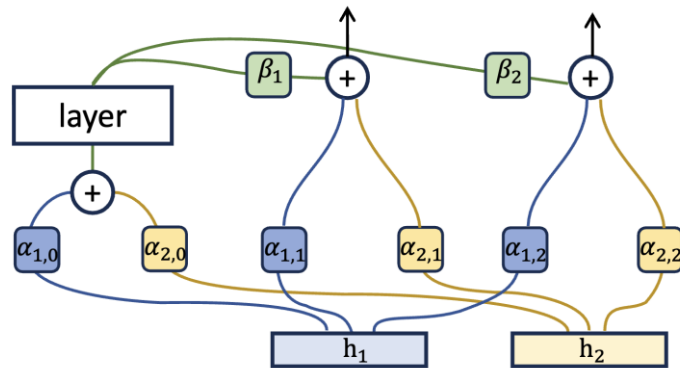
Skean et al., ICML 2025, arXiv:2502.02013 · La d et al., 2024, arXiv:2406.19384



Hyper-connections widen the bus without widening the layer

n parallel residual streams. The sublayer still runs at width d — only the plumbing changes.

- Pre mapping collects **n streams into one vector** of width d for the sublayer
- Post mapping **distributes** the output **back across the n streams**
- Res mapping **mixes the streams** between layers: a learned $n \times n$
- FLOPs barely move: 13.36G $\rightarrow$ 13.38G per token on a 7B OLMo MoE



(b) Hyper-connections

mHC constrains the mixing so depth cannot amplify

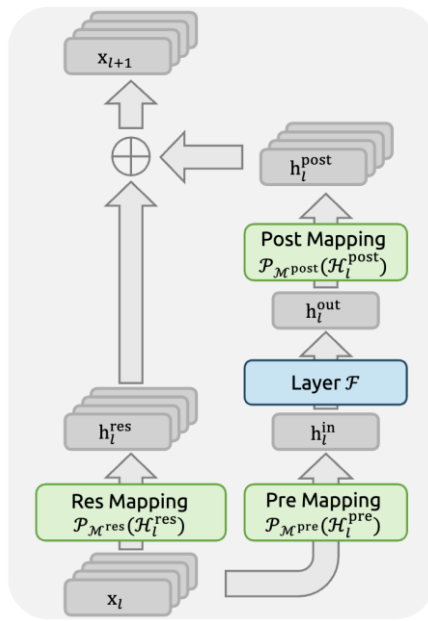
THE CONSTRAINT

- Res mapping is projected onto doubly stochastic matrices: rows and columns sum to 1
- Sinkhorn does the projection — 20 normalisation passes, visible in the config
- Pre and post mappings are non-negative and bounded, so reads and writes cannot cancel

WHERE IT SHIPPED

- DeepSeek V4: mHC with $n = 4$, +6.7% training time at 27B with fusion and recompute
- V4.1-Flash: Single-Pass mHC with a Mega-mHC kernel — the same idea, cheaper
- GLM-5.3-Flash adopted a four-stream mHC path too — so it is two labs now, not one

$$\mathcal{P}_{\mathcal{M}^{\text{res}}}(\mathcal{H}_l^{\text{res}}) := \{\mathcal{H}_l^{\text{res}} \in \mathbb{R}^{n \times n} \mid \mathcal{H}_l^{\text{res}} \mathbf{1}_n = \mathbf{1}_n, \mathbf{1}_n^\top \mathcal{H}_l^{\text{res}} = \mathbf{1}_n^\top, \mathcal{H}_l^{\text{res}} \geq 0\},$$



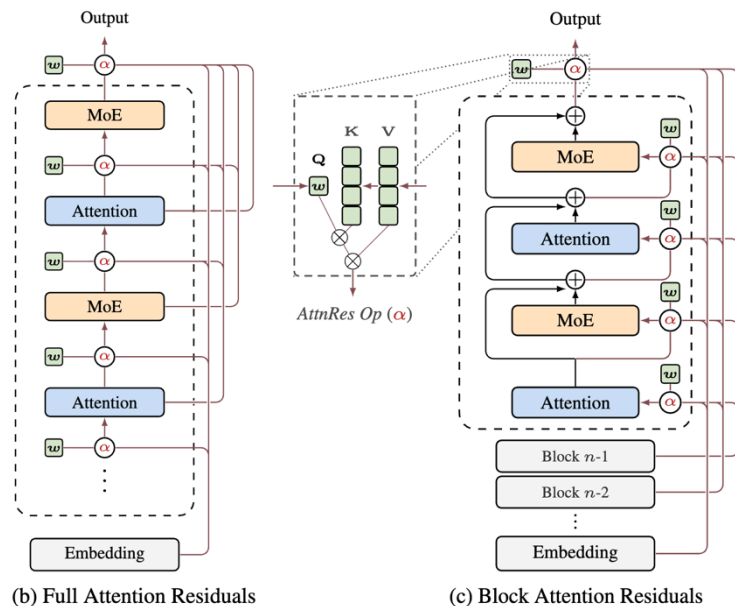
AttnRes lets a layer read depth by attention, not by sum

WHAT KIMI K3 CHANGED

- Learned pseudo-queries produce attention weights over the embedding and all previous block outputs
- So a layer selects what to read from history instead of taking the running sum
- Additivity of the residual stream survives inside a block, not across the model

THREE ANSWERS TO ONE QUESTION

- DeepSeek: more streams, constrained to a manifold
- Qwen: four branches with data-dependent read and write gates
- Kimi: one stream, but the read operation itself is replaced



2024-09

Hyper-Connections

n parallel streams, free mixing matrix

2025-12

mHC paper

mixing constrained to doubly stochastic

2026-04

DeepSeek V4

n = 4 streams in a 1.6T flagship

2026-07

Kimi K3 · AttnRes

layer reads depth by softmax, not by sum

2026-08

Qwen3.8-Flash-Next

Gated Residual: 4 branches, read/write gates



05

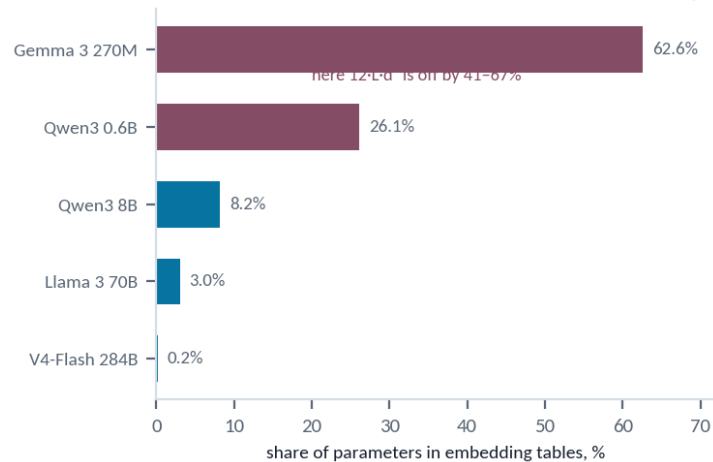
- LM.head()



Below two billion parameters, the tables are the model

The head is $V \times d$ and does not scale with depth, so its share falls as the stack grows — and explodes when it shrinks.

- Gemma 3 270M: 62.6% of all parameters sit in embedding tables
- Qwen3 0.6B: 26.1%, and that is already after weight tying
- Qwen3 8B: 8.2%. Llama 3 70B: 3.0%. A 284B MoE: 0.2%
- It is also the most numerically delicate matrix — logits usually get computed in fp32



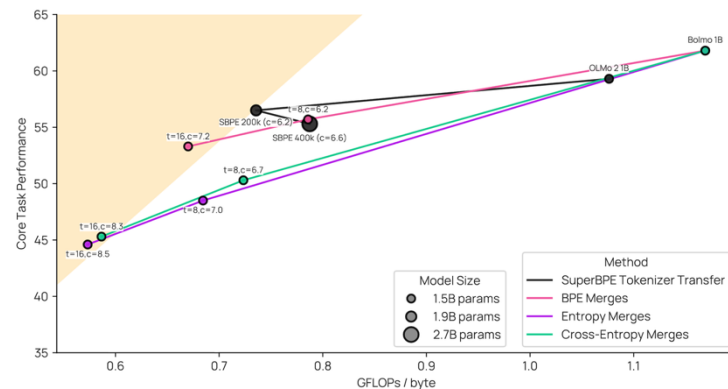
наш расчёт по публичным конфигам, 09.2026



A bigger vocabulary speeds you up, until softmax eats it

Fewer tokens per text means fewer forward passes. But every pass pays $V \cdot d$, and that term grows with V .

- So throughput rises with vocabulary size, reaches an optimum, and then falls
- Measured directly: the subword model leaves the Pareto frontier once softmax cost dominates
- Byte-level models take over the frontier at that point — they have no vocabulary to grow
- Bolmo 7B decodes at ~125 bytes/s against ~150 for its subword source at matched compression



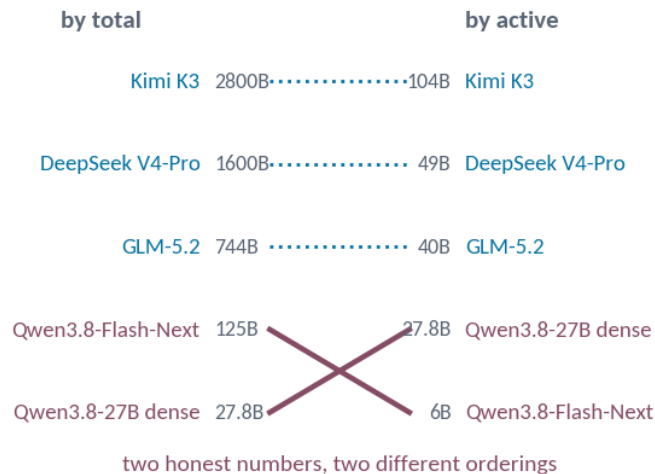
Ai2, Bolmo: Byteifying the Next Generation of Language Models, arXiv:2512.15586, Fig. 3



Twelve d squared per layer, plus the vocabulary tables

$\approx 12 \cdot L \cdot d^2 + (1 \dots 2) \cdot V \cdot d$ — the second term doubles when embeddings are not tied.

- Qwen3 8B: $12 \cdot 36 \cdot 4096^2 = 7.25B$ plus 0.62B tables $\rightarrow 7.87B$ against a real 7.57B
- Off by 4% there, and all of it is FFN shape and GQA
- Qwen3 0.6B is off by 41% and Gemma 3 270M by 67% — the tables again
- Report total and active separately: «35B-A3B» is not marketing, it is the honest format



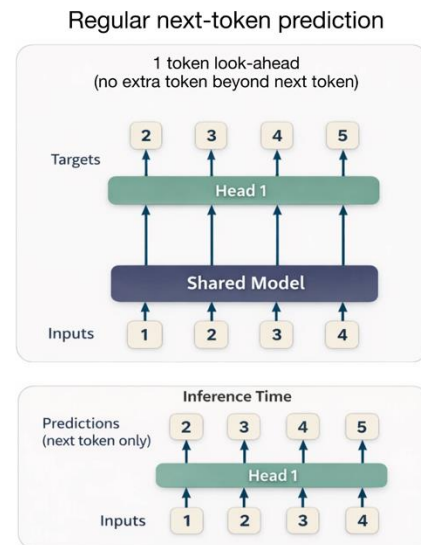
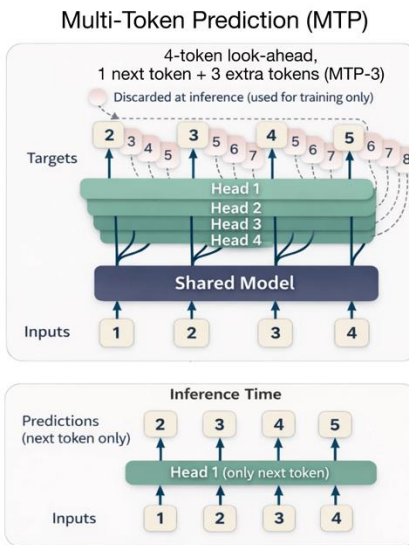
MTP: one head trained, two different things shipped

THE TRAINING OBJECTIVE

- Position t is asked about $t+1$ as usual, and about $t+2$ and $t+3$ through extra depths
- $L_{\text{next}} + (\lambda/D) \cdot \sum_k L_{\text{MTP}^{(k)}}$ — a denser learning signal per token
- The main decoder stays causal and still emits one token per step

THE ARTEFACT IT LEAVES BEHIND

- The extra path can be dropped after training — or reused as a draft model
- V4.1-Flash ships DSpark: three blocks over a 128-token window, five positions proposed at once
- How much that actually accelerates anything: lecture 12, with the engine named



06

- **Four tech reports, read by axis**

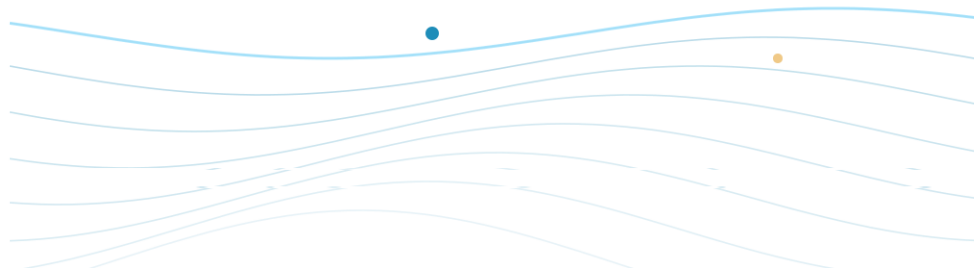
DeepSeek V4.1-Flash, Kimi K3, Qwen3.8-Flash-Next, GLM-5.3-Flash



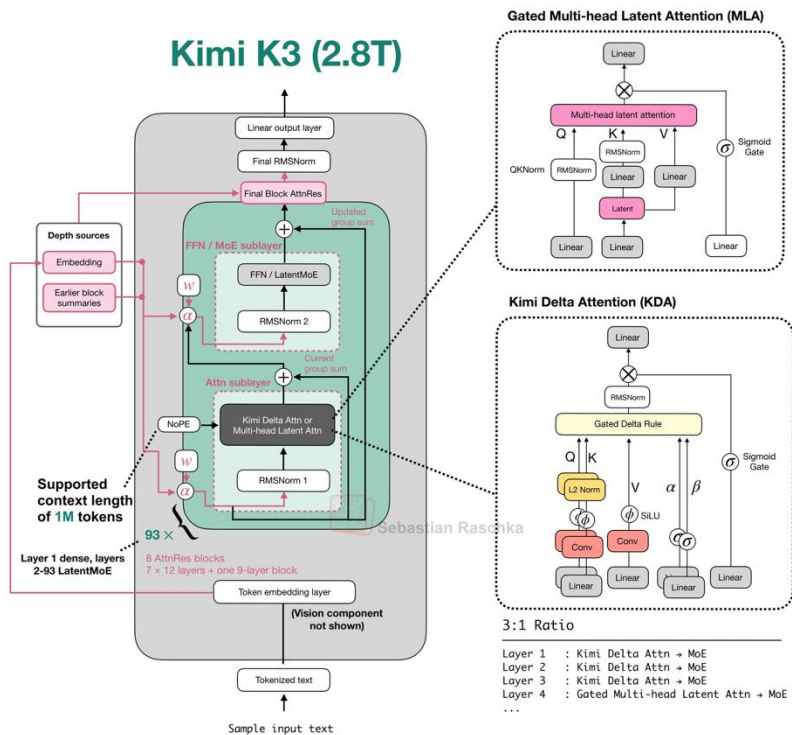
Everything inside the block changed; its shape did not

Decoder-only, an additive stream, a next-token loss. Everything else has been replaced at least once.

- Axis 1 — attention and KV: MHA → GQA → MLA → compressed, sparse, hybrid
- Axis 2 — sparsity: dense → MoE → fine-grained, shared, near-atomic experts
- Axis 3 — norm: Post-LN → Pre-RMSNorm → sandwich, QK-norm, QK-clip
- Axis 4 — the highway: one stream → several, gated, constrained, or read by attention



Kimi K3 scales information flow along three axes at once



Kimi K3 scales information flow along three axes at once

SEQUENCE AND DEPTH

- Three Kimi Delta Attention layers to one gated MLA per block
- The MLA layers run NoPE — no positional encoding at all, KDA decay carries recency
- So there is no θ base to retune and no YaRN when extending context

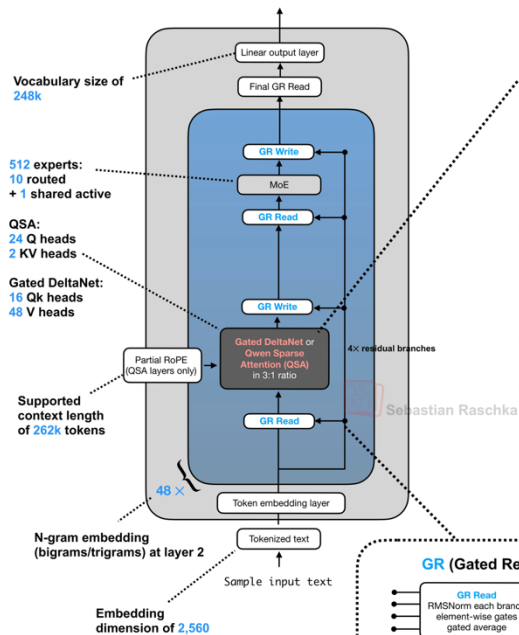
WIDTH

- Stable LatentMoE: 896 routed experts, 16 active, plus two shared
- Routed experts work in a narrow latent space — MLA's trick, one level up
- SiTU-GLU and quantile balancing hold optimisation together at that sparsity



Qwen3.8-Flash-Next

Qwen3.8-Flash-Next (125B-A6B)



Qwen Sparse Attention (QSA)

	Block 1				Block 2				Block 3			
	The	model	learns	to	attend	selectively	to	blocks	of	past	context	tokens
The	1	0	0	0	0	0	0	0	0	0	0	0
model	1	1	0	0	0	0	0	0	0	0	0	0
learns	1	1	1	0	0	0	0	0	0	0	0	0
to	1	1	1	1	0	0	0	0	0	0	0	0
attend	1	1	1	1	1	0	0	0	0	0	0	0
selectively	1	1	1	1	1	1	0	0	0	0	0	0
to	1	1	1	1	1	1	1	0	0	0	0	0
blocks	1	1	1	1	1	1	1	1	0	0	0	0
of	1	1	1	1	0	0	0	0	0	1	0	0
past	0	0	0	0	0	1	1	1	1	1	1	0
context	1	1	1	1	0	0	0	0	0	1	1	1
tokens	0	0	0	0	1	1	1	1	1	1	1	1

The current token attends to every token in selected complete micro-blocks, plus visible tokens in its current incomplete block

GR (Gated Residual)



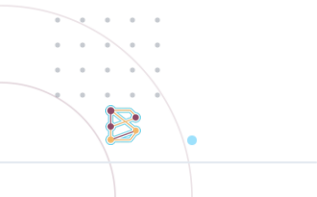
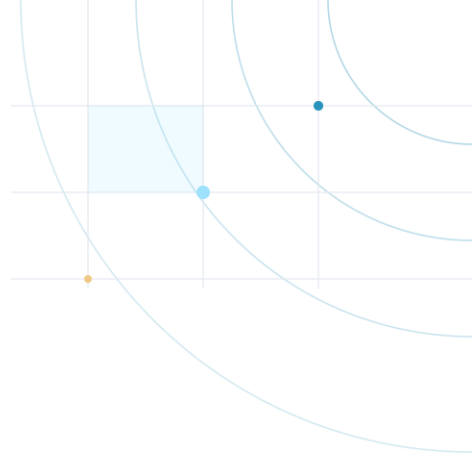
with 4 parallel residual branches

GR Read collapses the branches to one sublayer input
GR Write injects the output into all branches; there is no H_{res} branch mixer as in mHC

Qwen3.8-Flash-Next

Twelve repeats of $3 \times (\text{Gated DeltaNet} \rightarrow \text{MoE}) + 1 \times (\text{QSA} \rightarrow \text{MoE})$, hidden size 2560.

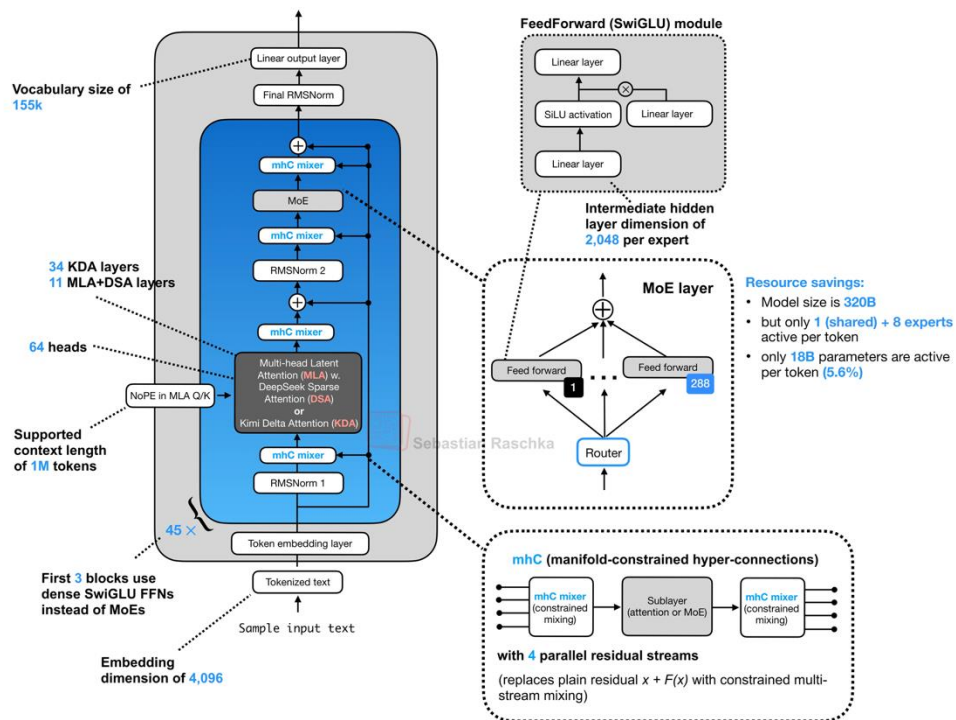
- QSA selects micro-blocks, not individual tokens — the indexer itself gets too expensive otherwise
- MoE: 512 experts of width 640, top-10 plus one shared — near-atomic experts
- Gated Residual: four widened branches, with data-dependent read and write gates
- Aa preview of the Qwen4 architecture



GLM-5.3-Flash

GLM-5.3-Flash (320B-A18B) (“Ox Alpha”)

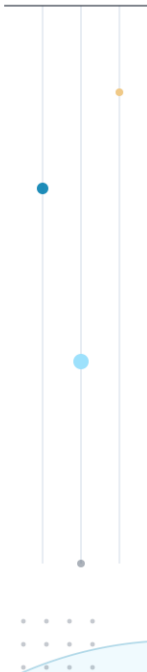
(Text decoder shown; 24-layer vision encoder and one MTP prediction layer omitted)



GLM-5.3-Flash is the hybrid one; GLM-5.3 is post-training only

Two releases with almost the same name, and only one of them is an architecture story.

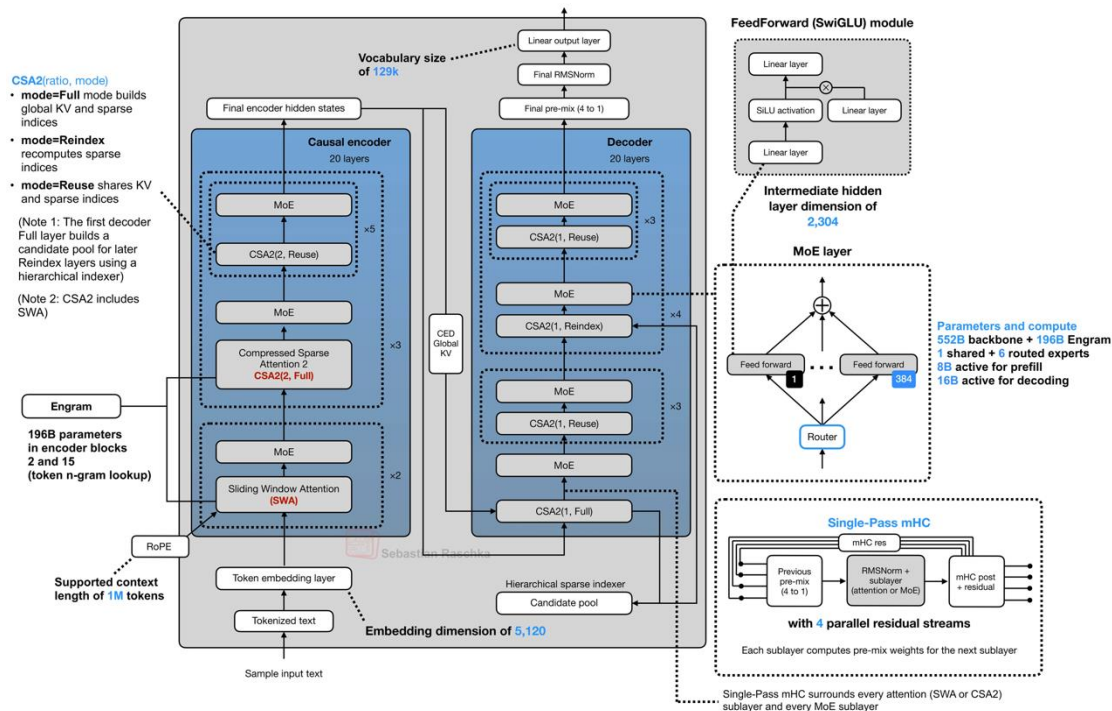
- GLM-5.3: the GLM-5.2 base reused exactly — 744B-A40B, every gain from post-training
- GLM-5.3-Flash: a new base, scaled down to 320B-A18B
- First GLM with sparse and linear attention together: KDA alongside MLA/DSA
- And a four-stream mHC residual path — the DeepSeek design, adopted by a second lab



Everything inside the block changed; its shape did not

DeepSeek V4.1-Flash (552B)

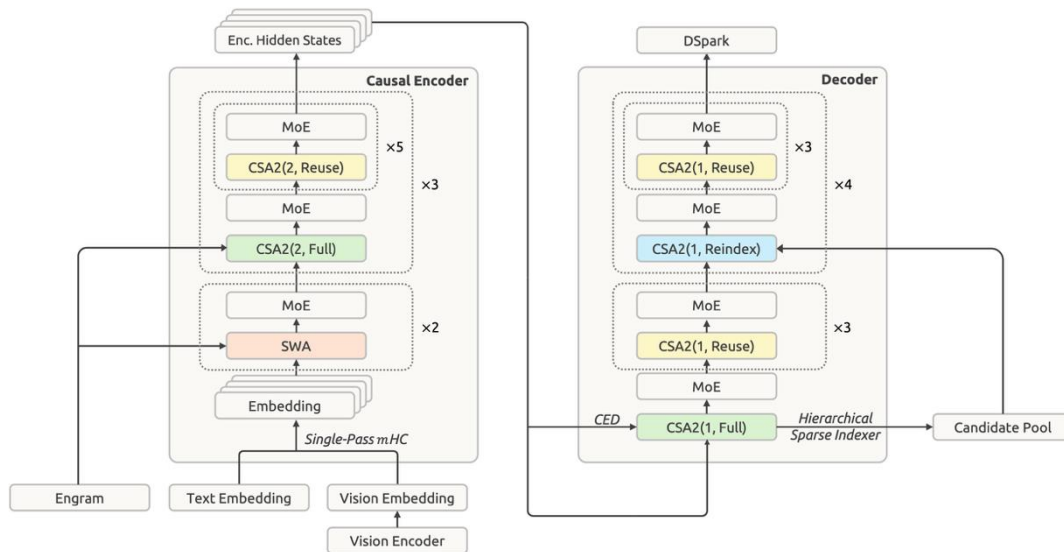
Text backbone shown. Vision encoder and DSpark speculative decoding modules omitted.



V4.1-Flash splits the stack into an encoder and a decoder

Causal Encoder-Decoder. The decoder's global KV is projected from the final encoder states, not from each decoder layer.

- 552B backbone; 8B active per prompt token, 16B per output token
- So input-heavy agentic traffic costs half of what generation costs
- KV down to 890 bytes per token — about a quarter of V4-Flash
- CSA2 gives each attention layer one of three modes: Full, Reindex, Reuse



Three of its components are exactly today's three blocks

WHAT IT ADDED

- Engram: 196B in two conditional-memory modules at layers 1 and 14
- n-grams of length 2, 3 and 4, eight hash heads, prefetched from host memory
- Single-Pass mHC with a Mega-mHC kernel; DSpark as the speculative decoder

WHAT THE REPORT SAYS

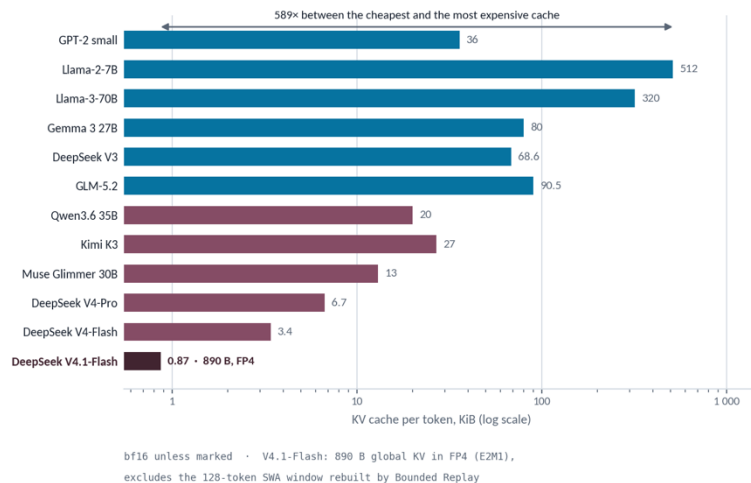
- CSA2 can select the wrong sparse positions — the authors name this themselves
- Bounded Replay approximates the local state it discarded
- The 196B of Engram are stored, not activated — do not add them to «active»



KV per token now spans two orders of magnitude

Same context, same year, 128× between the cheapest and the most expensive cache.

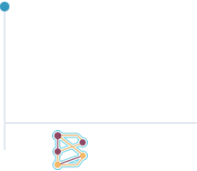
- Llama-2 7B kept 512 KiB per token; DeepSeek V4-Pro keeps 6.7 KiB
- V4.1-Flash goes further: 890 bytes, with FP4 main KV and SWA Bounded Replay
- A dense 30B can reach a frontier MoE's cache ratio — sparsity is not what buys it
- And the axis is not one-way: Qwen3.8 widened its KV against Qwen3.6



Four reports, four axes, no winner



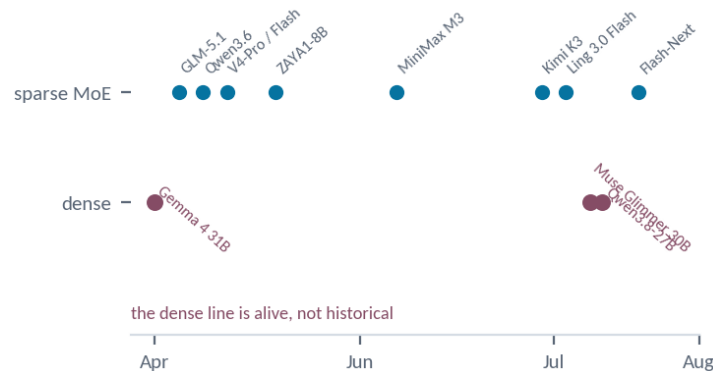
Model	Attention / KV	Sparsity	Norm	Highway
DeepSeek V4.1-Flash	CED + CSA2, 890 B	384, top-6 + 1	Pre-RMS + QK	Single-Pass mHC, n = 4
Kimi K3	KDA + gated MLA 3:1	896, 16 active	Pre-RMS, QK-clip	AttnRes over depth
Qwen3.8-Flash-Next	GDN + QSA 3:1	512, top-10 + 1	Pre-RMS + QK	Gated Residual, 4
GLM-5.3-Flash	KDA + MLA/DSA	~256, top-8 + 1	Pre-RMS	mHC, 4 streams
Qwen3.8-27B	GQA, dense attention	dense	Pre-RMS	single stream
Pangu Pro MoE	GQA	grouped, top-8	Pre-RMS	single stream



Dense is a choice too, and the teams state why

Every axis has models that deliberately stay at the base — with stated reasons, not by inertia.

- Qwen3.8-27B, 14 August: dense 27.8B — one GPU, 262k context, multimodal
- Google's stated reason for dense Gemma: MoE is harder to post-train, not worse
- Pangu: sparse, but balance solved in the architecture rather than in the loss
- On our scale dense is almost always the right default





The macro-architecture is coming apart on three of five points

holds: decoder-only + RMSNorm · moved: norm placement, positions, activations, sparsity, the highway



Eight things that were not obvious three hours ago

- | | | | |
|----|---|----|--|
| 01 | The FFN is two thirds of every block and never mixes positions — the cheap place to add capacity | 02 | Experts are geometry, not domains, and the balance loss actively suppresses the shared directions |
| 02 | Top-k has no gradient. Every balancing trick in the field exists to work around that one fact | 03 | Post-LN's gradients are not small, they are unbalanced by depth; Pre-LN pays for the fix with a growing residual |
| 02 | f is the load that happened, P is what the router intended; the loss multiplies them so load weights the gradient | 04 | The stream is a bus of fixed width, depth runs in stages, and the middle is the richest and the most redundant |
| 02 | Where you compute f matters more than which balancer you picked — micro-batch forbids specialization | 05 | Below two billion parameters the vocabulary tables are the model, and vocabulary size has an optimum |

And the one to remember: decoder-only and RMSNorm are all that is left of the consensus. The other five points have all moved since 2023.



HOMEWORK 1

Thirty of the hundred points are today's block

Extend a working decoder-only model, then plug it into Transformers. Everything runs on CPU.

Part	Pts	Where the material is
YaRN	15	Lecture 1, positions · seminar 1
Attention masks	15	Lecture 1 · seminar 1, the mask zoo
MLA	30	Lecture 1 · seminar 1, decoupled RoPE
Sparse MoE	30	Today, block 02 · seminar 2
Transformers integration	5	Seminar 2
Diagnostics and report	5	diagnostics.py, then report.md

Two weeks. Edit only `yarn.py`, `masks.py`, `mla.py`, `moe.py`, `hf_model.py` and `report.md` — tests are part of the spec. Run `check_submission.py` last: it checksums what you submit.



WHAT BLOCK 02 ALREADY ANSWERS

- Top-k routing and why the choice carries no gradient — slides 16 and 17
- The Switch loss $E \sum f_{e-p_e}$, and which factor is differentiable — slide 19
- Total against active parameters
- Capacity, dropping and re-normalization are the seminar's part



What to read and where to look



- 1 [DeepSeek-AI](#)
DeepSeek-V4.1-Flash — CED, Engram, Single-Pass mHC, DSpark
2026 · model card + tech report
- 2 [Kimi Team](#)
Kimi K3: Open Frontier Intelligence — KDA, AttnRes, LatentMoE
2026 · [arXiv:2607.24653](#)
- 3 [Zhu et al.](#) · [DeepSeek-AI](#)
Hyper-Connections · mHC: Manifold-Constrained Hyper-Connections
[arXiv:2409.19606](#) · [2512.24880](#)
- 4 [Lad, Gurnee & Tegmark](#)
The Remarkable Robustness of LLMs: Stages of Inference?
2024 · [arXiv:2406.19384](#)
- 5 [Skean et al.](#)
Layer by Layer: Uncovering Hidden Representations in Language Models
ICML 2025 · [arXiv:2502.02013](#)
- 6 [Dai et al.](#) · [Wang et al.](#)
DeepSeekMoE · Auxiliary-Loss-Free Load Balancing
[arXiv:2401.06066](#) · [2408.15664](#)
- 7 [Pangu Team](#)
Pangu Pro MoE: Mixture of Grouped Experts
2025 · [arXiv:2505.21411](#)
- 8 [Krajewski et al.](#)
Scaling Laws for Fine-Grained Mixture of Experts
ICML 2024 · [arXiv:2402.07871](#)
- 9 [Ai2](#)
Bolmo: Byteifying the Next Generation of Language Models
[arXiv:2512.15586](#)
- 10 [Geva et al.](#) · [Elhage et al.](#)
FFN as key–value memories · Transformer Circuits framework
[arXiv:2012.14913](#) · 2021



Lecture 3: Pre-training L data and scaling laws

seminar

today at 15:30

materials

course repository, slides go up today

homework

can be found on GitHub

reading

sign-up form in the chat



GitHub